

HyperAgent

The AI That Ships Code in a Bulletproof Sandbox

A Technical Deep-Dive | April 2026

Contents

1. The Problem Nobody Solved Clean
2. Architecture: Sandboxes All the Way Down
3. The MCP Bridge: Universal Tool Access
4. The Handler Model: Compile Once, Run Many
5. The Module System: A Standard Library
6. PDF Generation: Documents Without Dependencies
7. Running on Kubernetes
8. Security Model: Defense in Depth
9. Enhanced Capabilities Overview
10. What Can You Actually Build?
11. Looking Ahead

The Problem Nobody Solved Clean

Every AI coding assistant faces the same ugly dilemma: trust vs. capability.

Give the model shell access? You have handed a stochastic system root privileges. Restrict it to generating text? Enjoy syntactically plausible but untested hallucinations. Both extremes fail in production environments where security teams demand isolation and engineering teams demand real output.

The answer is a Hyperlight micro-VM: a hardware-isolated sandbox that spins up in microseconds, runs JavaScript (ES2023), and gives the AI exactly enough rope to build things without ever touching your host system.

No filesystem. No network. No `process.exit(1)`. No escape. And yet it builds PowerPoint decks, generates PDFs, creates ZIP archives, fetches web APIs, connects to any MCP-compatible tool, processes data, and writes files. All from inside the box. All at enterprise scale on Kubernetes.

HyperAgent represents a fundamental rethinking of how AI agents should interact with

computing resources. Instead of asking "how do we limit an all-powerful agent?" it asks "how do we give a constrained agent exactly the capabilities it needs?" The result is an architecture where security is not a layer bolted on top, but the foundation everything else is built upon.

Architecture: Sandboxes All the Way Down

The Micro-VM Layer

At the bottom of the stack sits Hyperlight, Microsoft's open-source micro-VM framework. Not containers. Not V8 isolates. Actual hardware-virtualized micro-VMs that boot in under a millisecond. The guest environment is radically minimal:

- No filesystem: there is no /tmp, no fs, no process
- No network: fetch() does not exist, XMLHttpRequest is unavailable
- No timers: setTimeout is not available
- Bounded resources: CPU timeout (1s), wall-clock (5s), heap (16MB), scratch (16MB)

This is not a sandbox with guardrails. It is a clean room with an airlock.

— HyperAgent Architecture Design Doc

The Plugin System: Controlled Escape Hatches

Plugins are host-side capabilities surgically injected into the sandbox as importable modules. Each plugin undergoes security auditing, requires explicit user approval, exposes a minimal API via host: imports, and is jailed to specific boundaries.

Plugin	Capability	Security Boundary
fetch	HTTPS network requests	Domain allowlist + SSRF protection + rate limiting
fs-read	Read filesystem	Jailed to single directory, read-only
fs-write	Write filesystem	Jailed to single directory, auto-enables fs-read

mcp	MCP tool bridge	Server allowlist + per-tool gating + audit logging
		The companion plugin pattern is elegant: enabling fs-write automatically enables fs-read because write-implies-read is a sane default. But fs-read alone? Read-only. Principle of least privilege, baked in at the architecture level.

The MCP Bridge: Universal Tool Access

The Model Context Protocol (MCP) is the emerging standard for connecting AI agents to external tools and data sources. The MCP bridge transforms HyperAgent from "an AI that builds documents" into "an AI that can do anything your tool ecosystem can do."

How It Works

1. Enable the MCP bridge plugin (requires explicit consent)
2. Configure which MCP servers HyperAgent can connect to
3. The bridge exposes MCP tools inside the sandbox as callable functions
4. Every invocation is logged and auditable

```
// Calling an MCP tool from inside the sandbox
import * as mcp from "host:mcp";
const result = mcp.callTool("github",
  "list-pull-requests",
  { repo: "hyperlight-dev/hyperagent",
    state: "open" }
);
```

What This Unlocks

- Query databases via MCP database servers
- Create Jira tickets via project-management servers
- Trigger deployments via CI/CD servers
- Search codebases via code-intelligence servers
- Pull metrics from Datadog, Grafana, or any monitoring system
- Interact with CRMs, ERPs, and internal tools

Critical design choice: The bridge does not give HyperAgent a blank check. You decide which MCP servers are reachable and which tools are callable. The AI proposes tool calls; the infrastructure gates them.

The Handler Model: Compile Once, Run Many

Code in HyperAgent is not "executed" directly. It is registered as named handlers, compiled, and then invoked with events. This enables stateful, multi-step workflows within a single sandbox session.

```
register_handler("my-builder", code)
// Compiled, NOT run
execute_javascript("my-builder",
  {action: "init"}) // NOW it runs
execute_javascript("my-builder",
  {action: "build"}) // State preserved
```

Module-level state persists across `execute_javascript` calls to the same handler. You get a tiny, stateful server inside the sandbox. The presentation object lives in module scope between calls: no serialization, no database, no temp files.

Registering, updating, or deleting any handler triggers recompilation of all handlers, clearing module-level state. For persistent state, use `ha:shared-state`.

State Management Patterns

Mechanism	Scope	Survives Recompile	Best For
Module-level vars	Single handler	No	In-progress document building
<code>ha:shared-state</code>	All handlers	Yes	Cross-handler data, config

The Module System: A Standard Library

The sandbox has nothing: no npm, no node_modules, no bundler. HyperAgent ships a curated set of system modules that implement hard things from scratch:

Module	Purpose
ha:str-bytes	String/byte conversion (replaces TextEncoder)
ha:crc32	CRC32 checksums for ZIP/PNG validation
ha:base64	Encode/decode without platform quirks
ha:xml-escape	Safe XML generation with element builder
ha:zip-format	Full ZIP archive creation with DEFLATE compression
ha:pptx	Complete PPTX generation with 8+ slide layouts
ha:pptx-charts	Bar, line, pie, combo, donut charts for PPTX
ha:pptx-tables	Tables, KV tables, timelines, comparisons
ha:ooxml-core	OOXML primitives, EMU conversions, color themes
ha:pdf	PDF generation: text, tables, images, graphics
ha:pdf-charts	Chart rendering for PDF documents
ha:doc-core	Themes, colour validation, contrast utilities
ha:shared-state	Cross-handler mutable state

There is a DEFLATE implementation running inside a 16MB-heap micro-VM that boots in microseconds. It powers the ZIP module, which powers the PPTX module, which generates valid Office documents with charts, themes, speaker notes, and transitions. The entire OOXML stack implemented as composable ES modules from scratch. No pptxgenjs. No officegen. No npm. Just math and bytes.

You can also register your own modules with `register_module`. They persist to disk across sessions and are importable by any handler. The module system is a singleton, enabling cross-handler communication without IPC.

PDF Generation: Documents Without Dependencies

The `ha:pdf` module produces real, valid PDF 1.7 documents entirely inside the micro-VM. No Puppeteer. No Chrome headless. No `wkhtmltopdf`. No 200MB binary dependencies. Pure JavaScript building the PDF binary format from scratch.

Full Capabilities

- Paragraphs, headings, rich text
- Tables with borders and styles
- Embed PNG/JPEG images
- Vector graphics and paths
- Auto-pagination engine
- Page numbers and footers
- Code blocks with monospace font
- Callout boxes and quote blocks
- Multiple professional themes
- Title pages and content pages
- Watermarks and metadata
- Charts (bar, line, pie, combo)

Flow Layout Engine

The PDF module features a flow-based layout engine that handles auto-pagination, margin management, and element positioning automatically. Elements are created using builder functions and flowed into the document with `addContent()`, which handles page breaks seamlessly.

```
import { createDocument, heading, paragraph,
  table, addContent, exportToFile } from "ha:pdf";
const doc = createDocument(
  { theme: "corporate-blue" });
const elems = [
  heading({ text: "Report", level: 1 }),
  paragraph({ text: "Revenue grew 21%..." }),
  table({ headers: ["Q", "Rev"], rows: [...] })
];
addContent(doc, elems); // auto-paginates!
exportToFile(doc, "report.pdf", fs);
```

This document you are reading right now was generated entirely inside a Hyperlight micro-VM using the `ha:pdf` module. Same data, two formats, one sandbox.

Running on Kubernetes: Micro-VMs Meet Orchestration

HyperAgent was born on a single machine. It now runs on Kubernetes with micro-VMs inside unprivileged pods. The micro-VMs themselves provide hardware-level isolation, so the pod needs no elevated privileges.

Traditional Approaches vs. Hyperlight

Approach	Isolation Level	Security Impact
Privileged container	VM-level	Bad for cluster security
Unprivileged container	Process-level	Weaker isolation guarantees
Separate pods per agent	Pod-level	Slow, expensive, poor scaling
Hyperlight micro-VMs	Hardware-level	No elevated privileges needed

What This Delivers

- No privileged: true needed in pod spec
- No hostPID, hostNetwork, or SYS_ADMIN capabilities
- Pod Security Standards compliant at restricted level (strictest tier)
- Multi-tenant safe: multiple instances on the same node, fully isolated
- Standard Kubernetes HPA/VPA horizontal scaling
- Near-zero overhead: micro-VMs are ephemeral and incredibly lightweight
- Enterprise-ready: IT teams deploy without bending security policies

This is the deployment model AI agents should have: secure enough for production, lightweight enough for scale, and boring enough for ops teams to approve without a three-month security review.

Security Model: Defense in Depth

HyperAgent implements 11 layers of security, each reinforcing the others:

- 1. Hardware virtualization via Hyperlight micro-VMs
- 2. No ambient capabilities (no fs, no net, no timers)
- 3. Plugin allowlisting with security audit + user approval
- 4. Domain allowlisting for fetch requests
- 5. MCP server allowlisting for tool access
- 6. Directory jailing for filesystem access
- 7. Resource bounding (CPU, wall-clock, heap, scratch, I/O)
- 8. SSRF protection built into the fetch plugin
- 9. Rate limiting on all external requests
- 10. Comprehensive audit logging for every action
- 11. Kubernetes-native isolation in restricted PSS pods

The AI can write and execute arbitrary JavaScript. And it still cannot read /etc/passwd, phone home, or mine crypto (1-second CPU timeout, 16MB heap). This is what "secure by default" looks like when you actually mean it.

Resource Profiles: One-Click Configuration

Instead of manually tweaking settings, profiles bundle everything. They stack additively:

Profile	Purpose	Configuration
default	Vanilla sandbox	No plugins, tight limits
file-builder	Document generation	fs-write enabled, bumped scratch
web-research	API calls	fetch with domain allowlisting
heavy-compute	Data crunching	High CPU/wall timeouts, large heap

Enhanced Capabilities Overview

Beyond the core architecture, HyperAgent delivers capabilities that make it a genuine production tool rather than a tech demo:

Document Generation

The PPTX stack supports 8+ slide layouts including title slides, content slides, chart slides, comparison layouts, and section headers. Each presentation uses professional themes with coordinated color palettes. Charts include bar, line, pie, combo, and donut variants. The PDF stack provides a complete flow-based layout engine with automatic pagination, tables, code blocks, callout boxes, rich text, images, headers, footers, page numbers, and watermarks.

Data Processing and Analysis

- CSV parsing and transformation with custom delimiters
- JSON reshaping, filtering, and aggregation
- Statistical analysis: means, medians, standard deviations
- Data visualization through charts in PPTX and PDF
- Cross-referencing data from multiple sources via MCP

Web Integration and Multi-Step Pipelines

The fetch plugin enables secure HTTPS requests to allowlisted domains for live data integration. Combined with document generation, HyperAgent handles complete workflows: (1) fetch live data from APIs, (2) process and analyze, (3) generate a presentation with charts, (4) generate a companion PDF, (5) deliver both artifacts. All in one session, all inside the sandbox.

What Can You Actually Build?

With this architecture, HyperAgent routinely builds real-world artifacts:

Artifact	Modules	Example
PowerPoint decks	ha:pptx + charts	Themed slides with charts, tables, images
PDF documents	ha:pdf + charts	Reports, proposals, invoices with tables
ZIP archives	ha:zip-format	Bundled deliverables with DEFLATE
Data analysis	Pure JS	CSV parsing, statistics, JSON reshaping
API integration	fetch plugin	Live data from Github, APIs, services
Tool automation	MCP bridge	Jira, CI/CD, databases, monitoring

The Developer Experience

1. Check what exists: `list_modules`, `list_plugins`
2. Read the docs: `module_info("pptx")`, `plugin_info("fetch")`
3. Set up resources: `apply_profile("file-builder")`
4. Register code: `register_handler("builder", code)`
5. Execute: `execute_javascript("builder", {action: "go"})`
6. Deliver: files land in the jailed output directory

No build step. No dependency resolution. No container pull. Register, run, done. Cold-start overhead is microseconds. When something goes wrong, errors are deterministic and tell you exactly what to fix.

Looking Ahead

HyperAgent is open source and actively evolving. The project welcomes contributions at github.com/hyperlight-dev/hyperagent.

Roadmap Highlights

- Custom font embedding for PDF (TrueType support)
- XLSX (Excel) generation module for spreadsheet output

- Enhanced chart types: scatter plots, area charts, Gantt charts
- Image manipulation: resize, crop, composite within the sandbox
- Streaming output for large document generation
- Additional MCP tool integrations and community bridges
- Performance profiling and optimization tooling

This blog post was conceived, structured, and delivered by the system it describes. The AI runs inside the Hyperlight sandbox, checking its own module list and writing about its own architecture from inside the box. This PDF was generated entirely within the same micro-VM.

— HyperAgent

HyperAgent: an AI that does real work in a real sandbox with real security guarantees, now at enterprise scale.

github.com/hyperlight-dev/hyperlight

github.com/hyperlight-dev/hyperagent

github.com/hyperlight-dev/hyperlight-on-kubernetes